Computer Organisation

Topic 3

Computer Arithmetic

(Part 2)

Stallings, Computer Organization & Architecture (9th Ed.), Chap. 10

Outline

1. Arithmetic & Logic Unit

2. Integer Representation
   - Sign-magnitude
   - Twos complement
   - Conversion between lengths

3. Integer Arithmetic
   - Addition & Subtraction
   - Multiplication
   - Division

4. Floating-point Arithmetic

2's complement operations

Addition

Subtraction

Multiplication

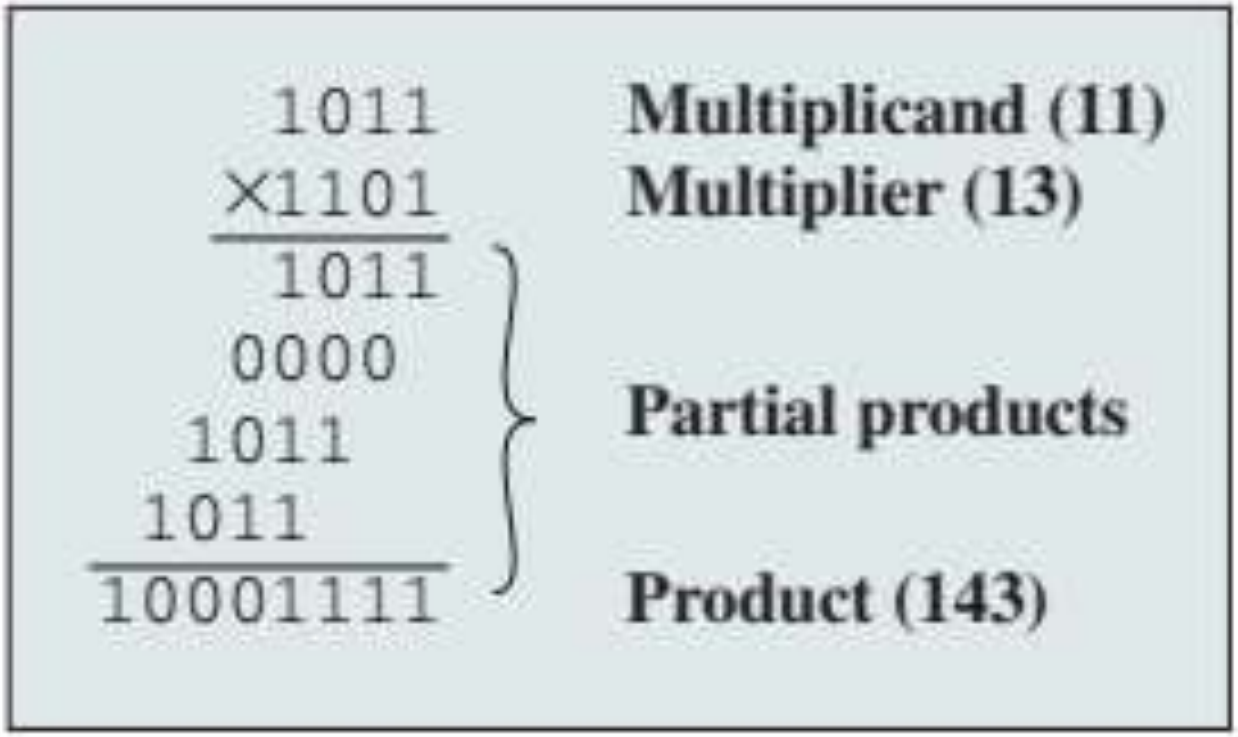
Division

Add-shift method (unsigned integer)
Booth's algorithm (-ve numbers)

Multiplication

- Compared with addition and subtraction, multiplication is a **complex operation**, whether performed in hardware or software.
- A wide variety of algorithms have been used in various computers.
- Work out **partial product** for each digit
- Take care with place value (column)
- **Add partial products**

Multiplication of unsigned integer



11 x 13

- 11 = multiplicand
- 13 = multiplier

				1	0	1	1	11
			X	1	1	0	1	13
Partial products				1	0	1	1	
			0	0	0	0	x	Addition operation
		1	0	1	1	x	x	
	1	0	1	1	x	x	x	
1	0	0	0	1	1	1	1	

Note:

- if multiplier bit is 1 copy multiplicand (place value) otherwise zero
- Need double length result

Multiplication

- How can this be more efficient?

1. Perform running addition on partial products
   - Do not have to wait until the end
   - Fewer registers are needed

This eliminates the need for storage of all the partial products

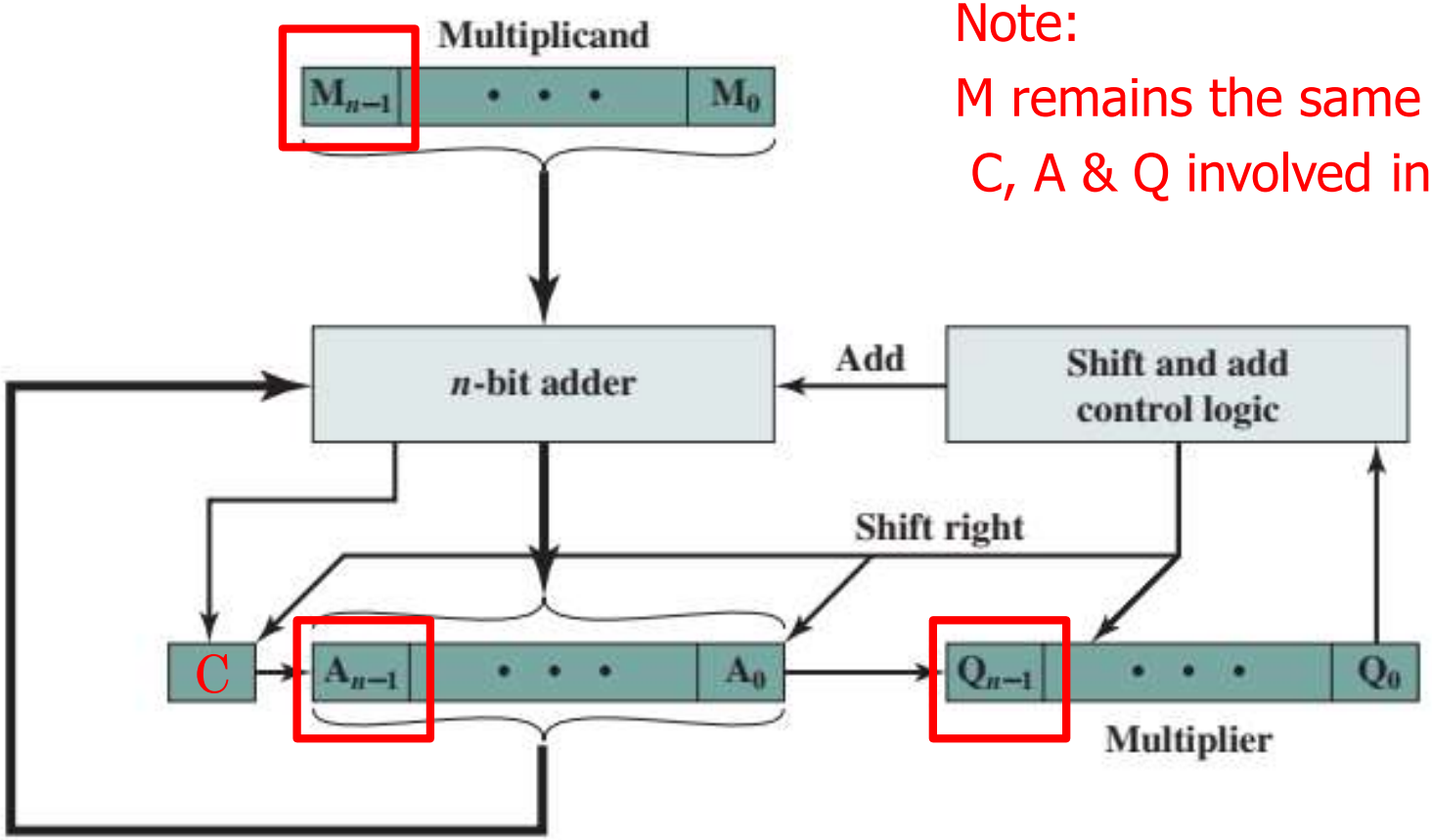
Multiplication

- How can this be more efficient?

2. Save time on generation of partial products
    i. For each 1 on multiplier requires an add and a shift operation
    ii. For each 0 on multiplier, only a shift is required

M: $(11)_{10}$
Q: $(13)_{10}$
A: (additional register)
C: Carry bit

Multiplication block diagram



Note:
M remains the same
C, A & Q involved in shifting

For each 1 on multiplier requires an add and a shift operation
For each 0 on multiplier, only a shift is required

Unsigned Binary Multiplication

M: Multiplicand = $(11)_{10}$

Q: Multiplier = $(13)_{10}$

A: (additional register)

C: Carry bit

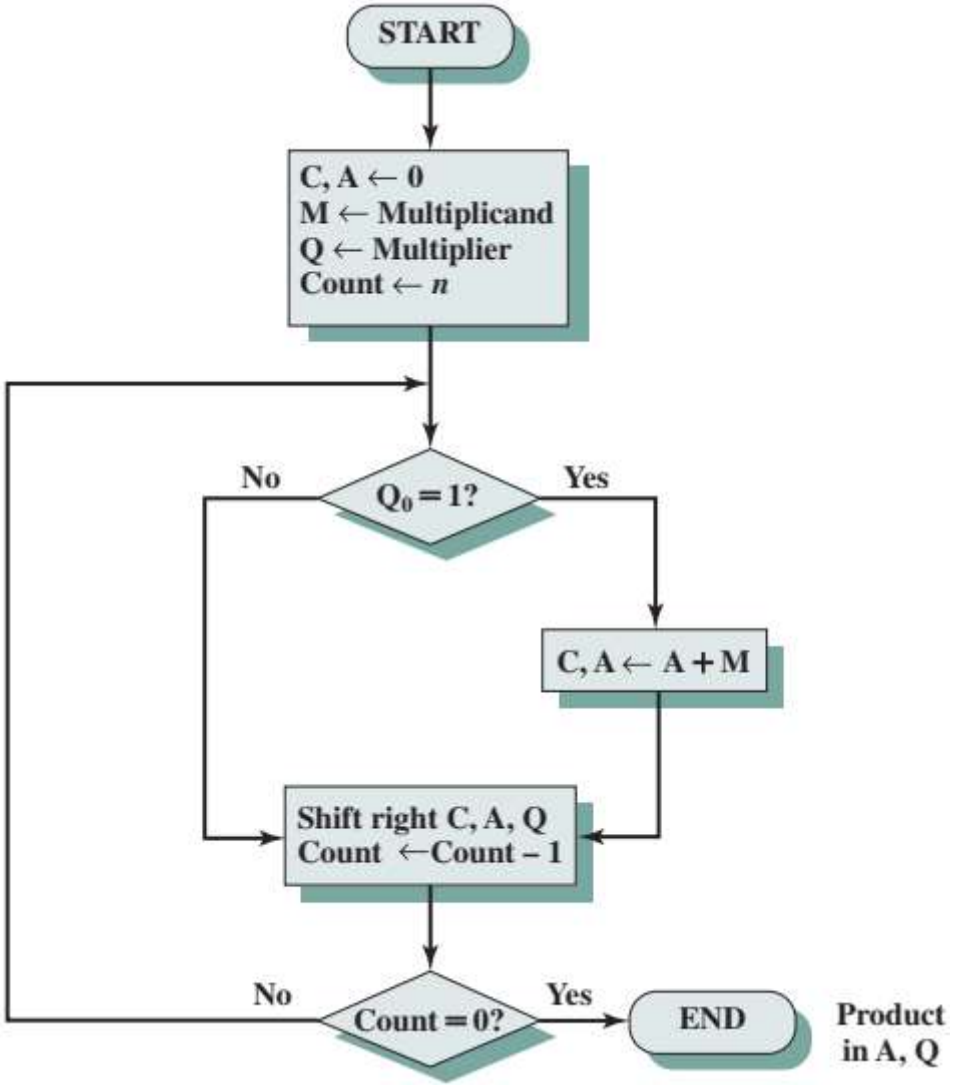
C	A	Q	M		
0	0000	1101	1011	**Initial values**	
0	1011	1101	1011	**Add**	**First cycle**
0	0101	1110	1011	**Shift**	
0	0010	1111	1011	**Shift**	**Second cycle**
0	1101	1111	1011	**Add**	**Third cycle**
0	0110	1111	1011	**Shift**	
1	0001	1111	1011	**Add**	**Fourth cycle**
0	1000	1111	1011	**Shift**	

Note:

M remains the same

C, A & Q involved in shifting

Flowchart for Unsigned Binary Multiplication



11 x 13

- 11 = multiplicand
- 13 = multiplier

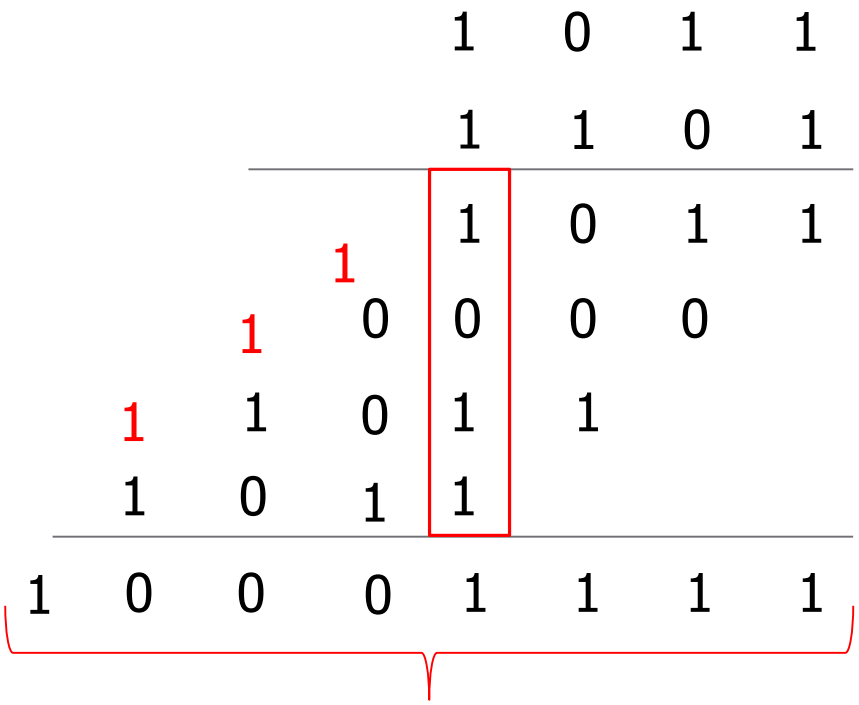
				1	0	1	1	11
			X	1	1	0	1	13
				1	0	1	1	
			0	0	0	0	x	
		1	0	1	1	x	x	
	1	0	1	1	x	x	x	
1	0	0	0	1	1	1	1	

$(11)_{10}$ x $(13)_{10}$

$(11)_{10} = (1011)_2$

$(13)_{10} = (1101)_2$

0 + 0 = 0
0 + 1 = 1
1 + 0 = 1
1 + 1 = 0 carry forward 1



$(13)_{10}$ x $(9)_{10}$

$(13)_{10} = (1101)_2$
$(9)_{10} = (1001)_2$

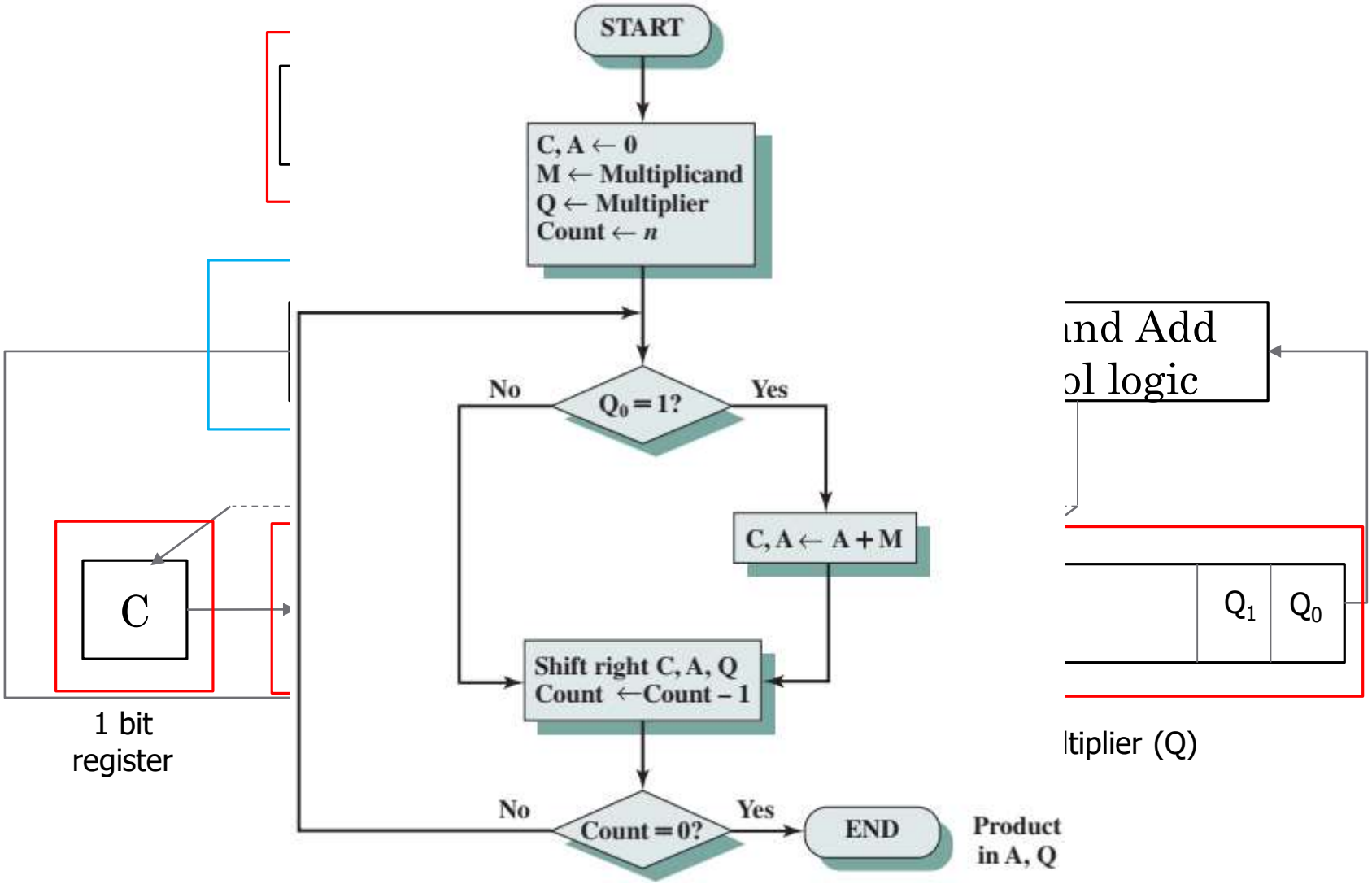
0 + 0 = 0
0 + 1 = 1
1 + 0 = 1
1 + 1 = 0 carry forward 1

			1	1	0	1	
			1	0	0	1	
		1	1	1	0	1	
		0	0	0	0		
	0	0	0	0			
1	1	0	1				
0	1	1	1	0	1	0	1

$= (117)_{10}$

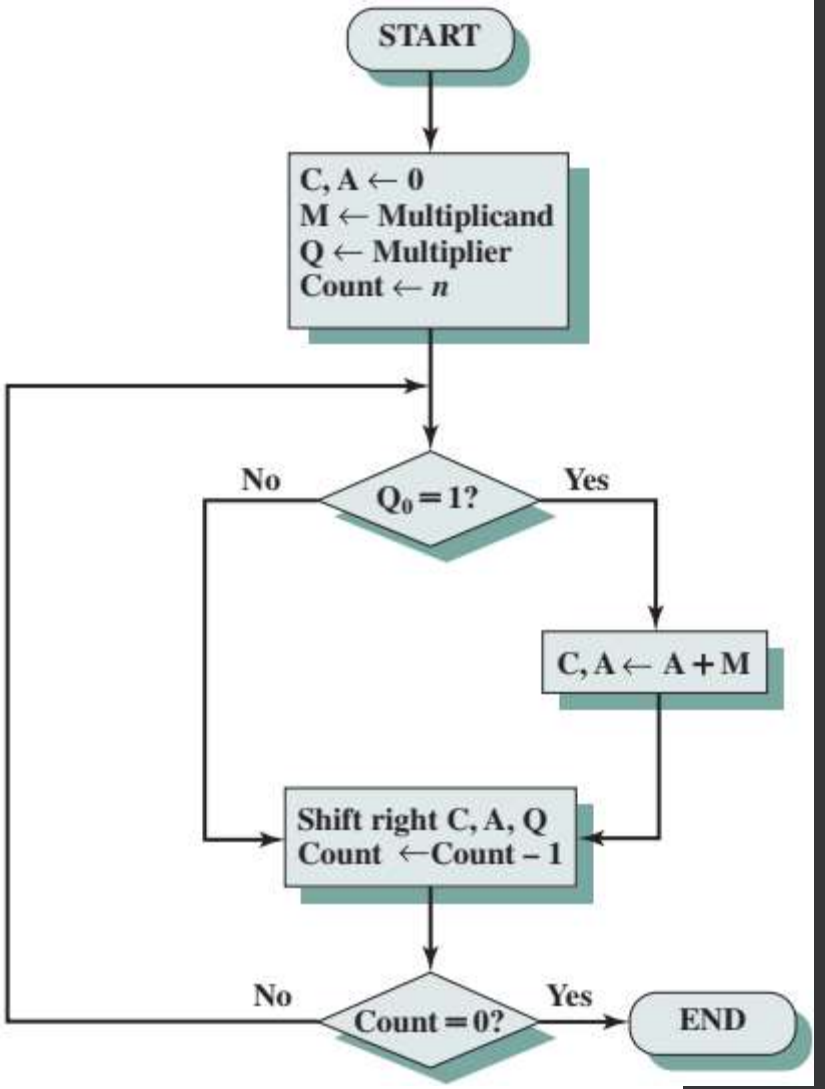
Multiplication using Add-shift method

- 11 x 13
- 11 = M (multiplicand)
- 13 = Q (multiplier)
- 143 = AQ



Hardware Structure for add-shift multiplication

1. Initialize C, A to 0
2. Fill the (M) and (Q) with the binary value
   - Count = bit value
3. N cycle:
   - Check the Q_0 value
     - ✓ If 1 = Add value in (A) and (M)
       - ➢ Shift RIGHT (C), (A) and (Q)
       - ➢ Count – 1
     - ✓ If 0 = Shift RIGHT (C), (A) and (Q)
       - ➢ Count -1
4. Final cycle = count = 0



13 ($M_{ultiplicand}$) = 1101
11 ($Q_{Multiplier}$) = 1011

13 x 11
Expected results:
$(143)_{10}$
$(10001111)_2$

C	A	Q	Operation	Cycle
0	0000	1011	Initial	
0	1101	1011	Add (A+M)	1st
0	0110	1101	Shift right	
1	0110	1101	Add (A+M)	2nd
0	1001	1110	Shift right	
0	0100	1111	Shift right	3rd
1	0001	1111	Add (A+M)	4th
0	1000	1111	Shift right	

Shift count = bit count
Add count = 1 count in Q

13 ($M_{ultiplicand}$) = 1101 (remain unchanged)
11 ($Q_{Multiplier}$) = 1011

n=4

13 x 11
Expected results:
$(143)_{10}$
$(10001111)_2$

C	A	Q	Operation	Cycle
0	0000	1 0 1 1	initialization	-
0 0	0000 + 1101 0 1 1 0	Q_3 Q_2 Q_1 Q_0 1 0 1 1 1 1 0 1	A+M Shift → C, A & Q	1st cycle
1 0	0011 + 1101 1 0 0 1 1	1 1 0 1 1110	A+M Shift → C, A & Q	2nd cycle
0	0100	1111	Shift → C, A & Q	3rd cycle
1 0	0100 + 1101 10001	1111 1111	A+M Shift → C, A & Q	4th cycle

12 ($M_{ultiplicand}$) = 1100
14 ($Q_{Multiplier}$) = 1110

12 x 14
Expected results:
$(168)_{10}$
$(10101000)_2$

C	A	Q	Operation	Cycle
0	0000	1110	Initial	
0	0000	0111	Shift C, A &Q	1st
0	1100	0111	Add A+M	2nd
0	0110	0011	Shift C, A &Q	
1	0010	0011	Add A+M	3rd
0	1001	0001	Shift C, A &Q	
1	0101	0001	Add A+M	4th
0	**1010**	**1000**	Shift C, A &Q	

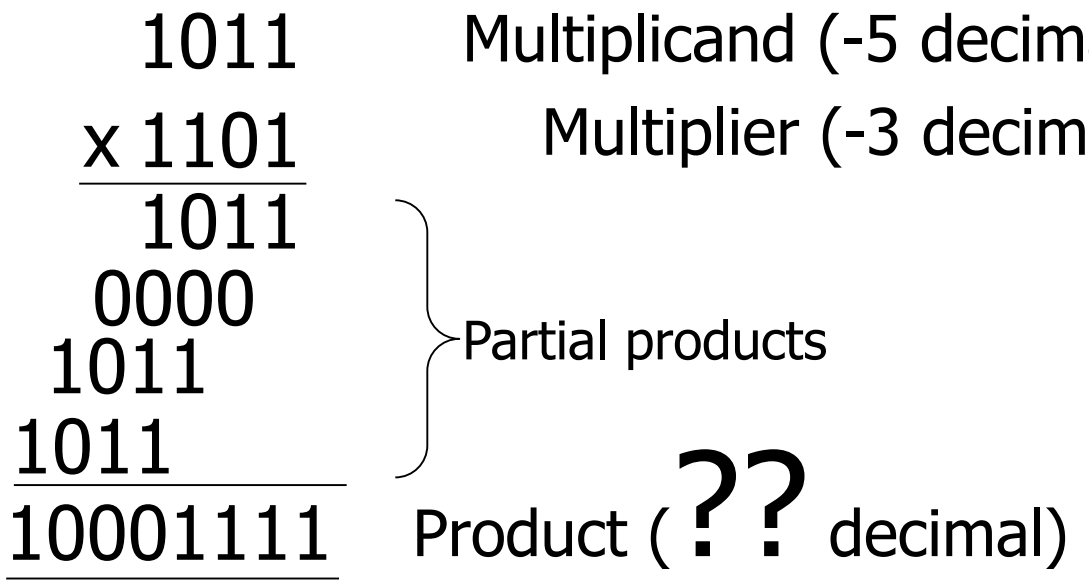
Quick activity #7

Multiply the following values using add-shift multiplication method:

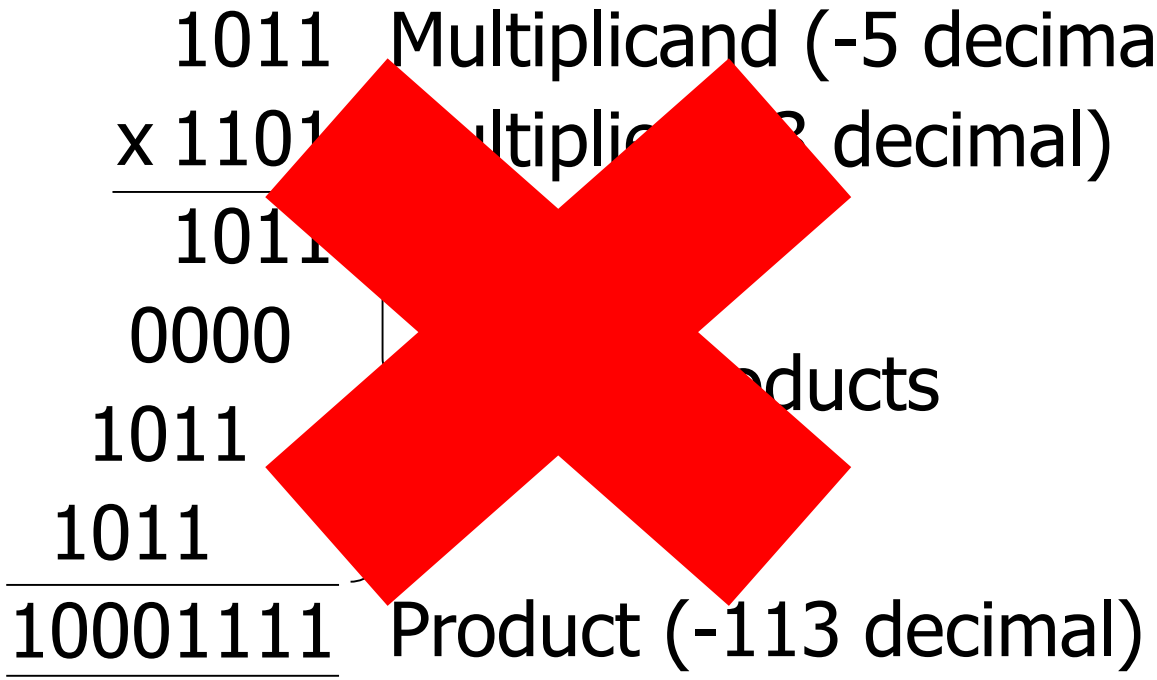
- 10 x 10
- 9 x 12

How we handle multiplication of two negative numbers?

Multiplying Negative Numbers (2s Complement)



Multiplying Negative Numbers (2s Complement)



Multiplying Negative Numbers (2s Complement)

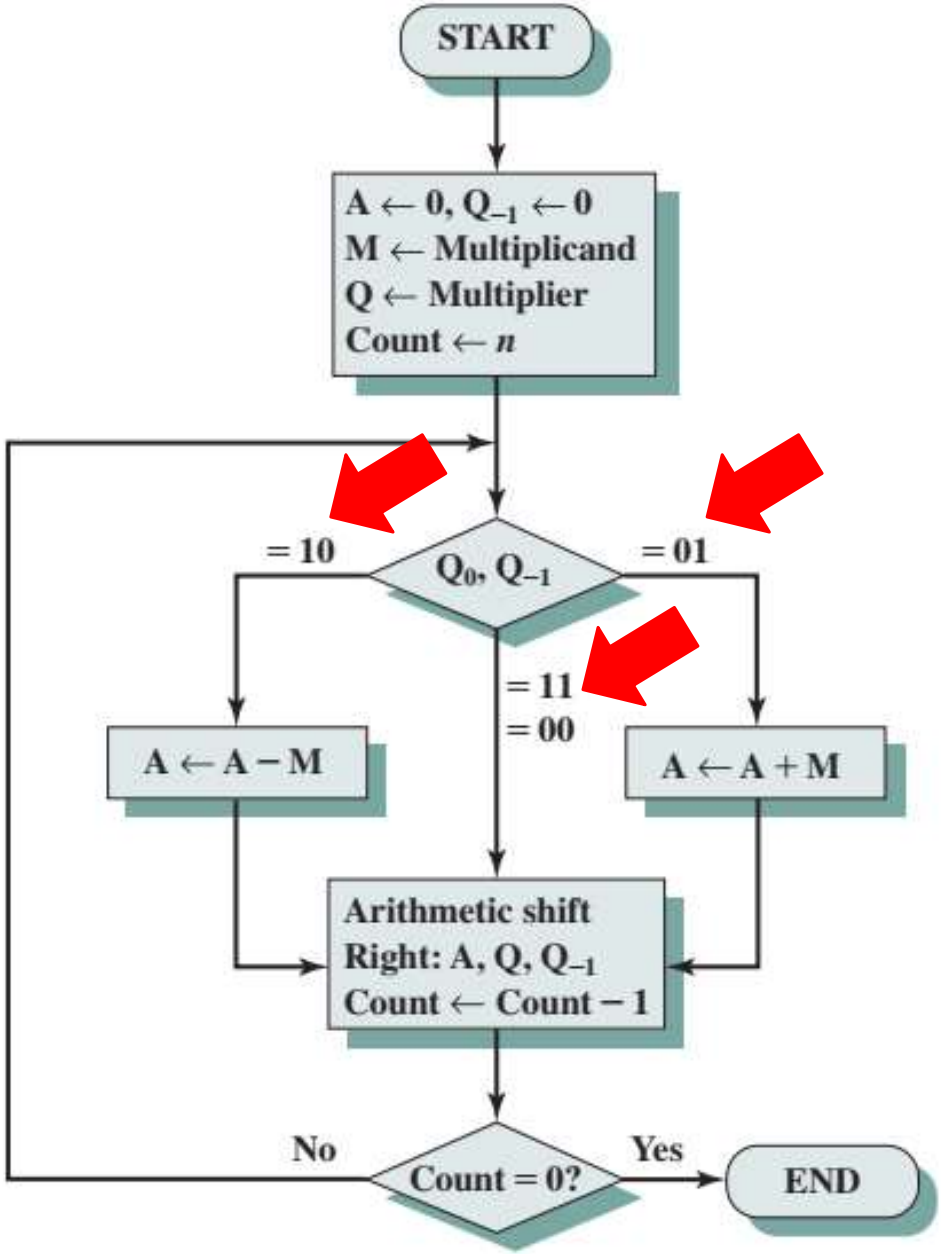
Solution 1

- **Convert to positive** if required
- Multiply as above
- **If signs were different, negate** answer
  - Implementers prefer techniques that do not require this final transformation step

Multi ıent)

Solu

- B
- M bers in 2s co
- In ı in 1950s



Booth's Algorithm conditions

$Q_3Q_2Q_1Q_0Q_{-1}$

Q_0	Q_{-1}	**Operation**
0	0	Shift right A, Q, Q_{-1}
1	1	
0	1	Add (A+M) Shift right A, Q, Q_{-1}
1	0	Subtract (A-M) Shift right A, Q, Q_{-1}

Booth's algorithm

7 x 3
Expected results:
$(21)_{10}$
$(10101)_2$

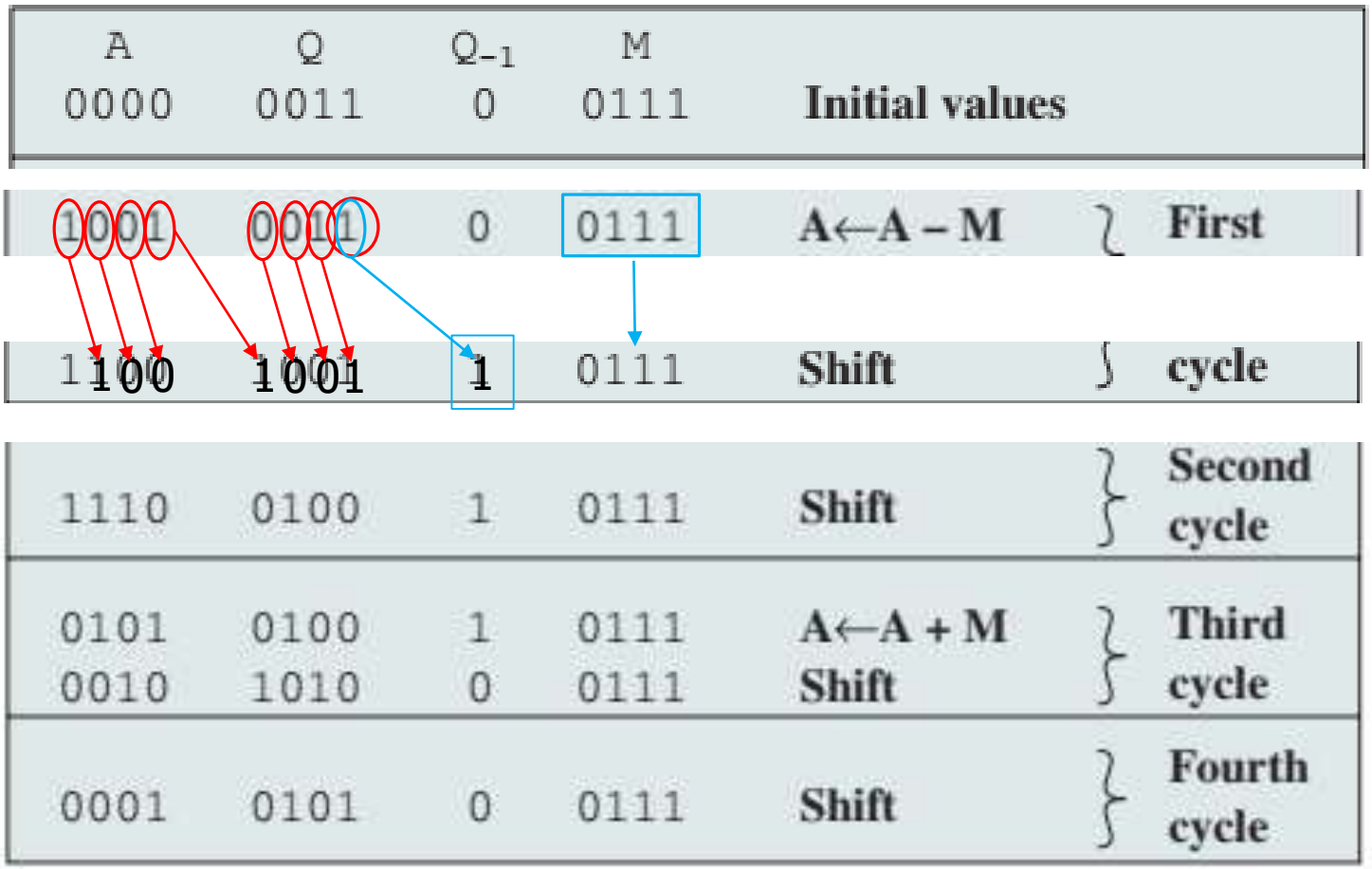
- 7 ($M_{ultiplicand}$) = 0111(remain unchanged)
- 3 ($Q_{Multiplier}$) = 0011

A	Q	Q_{-1}	M		
0000	0011	0	0111	**Initial values**	
1001	0011	0	0111	**A←A – M**	**First cycle**
1100	1001	1	0111	**Shift**	
1110	0100	1	0111	**Shift**	**Second cycle**
0101	0100	1	0111	**A←A + M**	**Third cycle**
0010	1010	0	0111	**Shift**	
0001	0101	0	0111	**Shift**	**Fourth cycle**

Booth's algorithm

- 7 ($M_{ultiplicand}$) = 0111(remain unchange
- 3 ($Q_{Multiplier}$) = 0011

Q_0	Q_{-1}	Operation
0	0	Shift right A, Q, Q_{-1}
1	1	
0	1	Add (A+M) Shift right A, Q, Q_{-1}
1	0	Subtract (A-M) Shift right A, Q, Q_{-1}



A	Q	Q_{-1}	M		
0000	0011	0	0111	Initial values	
1001	0011	0	0111	A←A – M	First
1100	1001	1	0111	Shift	cycle
1110	0100	1	0111	Shift	Second cycle
0101	0100	1	0111	A←A + M	Third
0010	1010	0	0111	Shift	cycle
0001	0101	0	0111	Shift	Fourth cycle

Booth's Algorithm

Example 2

7 ($M_{ultiplicand}$) = 0111
-6 ($Q_{Multiplier}$) = 1010

2s M = 1001

Q_0	Q_{-1}	Operation
0	0	Shift right A, Q, Q_{-1}
1	1	
0	1	Add (A+M) Shift right A, Q, Q_{-1}
1	0	Subtract (A-M) Shift right A, Q, Q_{-1}

A	Q	Q_{-1}	M	Operation	Cycle
0000	1010	0	0111	Initialize	
0000	0101	0	0111	Shift	1st cycle
1001	0101	0	0111	A-M = A+2sM	2nd cycle
1100	1010	1	0111	Shift	
0011	1010	1	0111	A+M	3rd cycle
0001	1101	0	0111	Shift	
1010	1101	0	0111	A-M = A+2sM	4th cycle
1101	0110	1	0111	Shift	

Booth's Algorithm

Example 3

Case 1: 4 x 4

M = 4 = 0100

Q = 4 = 0100

2s M = 1100

A	Q	Q_{-1}	Operation	Cycle
0000	0100	0	Init.	
0000	0010	0	Shift	
0000	0001	0	Shift	
1100 1110	0001 0000	0 1	A-M Shift	
0010 0001	0000 0000	1 0	A+M Shift	

0000
1100
1100

1110
0100
10010

Case 1: 4 x -4

M = 4 = 0100 2s M = 1100

Q = -4 = 1100 2s Q = 0100

A	Q	Q_{-1}	Operation	Cycle
0000	1100	0	Init.	
0000	0110	0	Shift	
0000	0011	0	Shift	
1100 1110	0011 0001	0 1	A-M Shift	
1111	0000	1	Shift	

0000
1100
1100

Case 1: -4 x 4

M = -4 = 1100 2s M = 0100

Q = 4 = 0100

A	Q	Q_{-1}	Operation	Cycle
0000	0100	0	Init.	
0000	0010	0	Shift	
0000	0001	0	Shift	
0100 0010	0001 0000	0 1	A-M Shift	
1110 1111	0000 0000	1 0	A+M Shift	

0000
0100
0100

0010
1100
1110

Case 1: -4 x -4

M = -4 = 1100 2s M = 0100

Q = -4 = 1100 2s Q = 0100

A	Q	Q_{-1}	Operation	Cycle
0000	1100	0	Init.	
0000	0110	0	Shift	
0000	0011	0	Shift	
0100 0010	0011 0001	0 1	A-M Shift	
0001	0000	1	Shift	

0000
0100
0100

Advantages of Booth's Algorithm

- Performed signed multiplications
- Performs better of –ve numbers are involved
- Reduced number of operations

Quick activity #8

1. Use the Booth algorithm to multiply 23 (multiplicand) by 29 (multiplier), where each number is represented using 6 bits
2. Use the Booth algorithm to multiply 10 (multiplicand) by -10 (multiplier), where each number is represented using 5 bits
3. Use the Booth algorithm to multiply -6 (multiplicand) by -5 (multiplier), where each number is represented using 4 bits
4. Use the Booth algorithm to multiply -7 (multiplicand) by 5(multiplier), where each number is represented using 4 bits
5. Given x = 0100 and y = 1100 in twos complement notation (i.e., x = 4, y = -4), compute the product $p = x * y$ with Booth's algorithm

2's complement operations

Addition

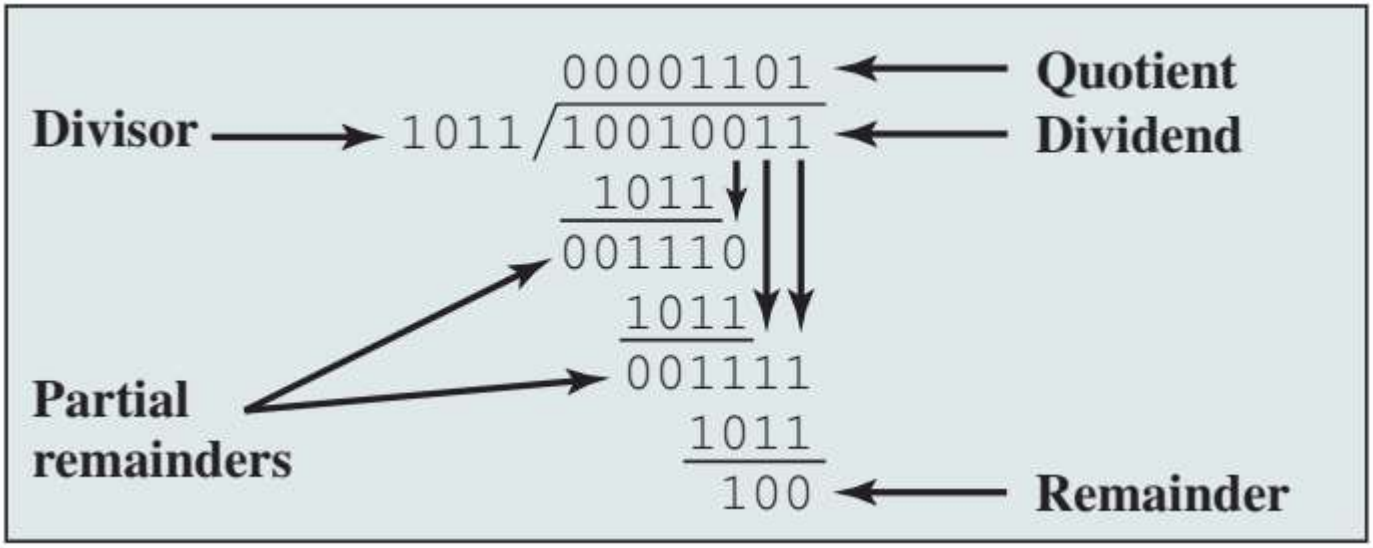
Subtraction

Multiplication

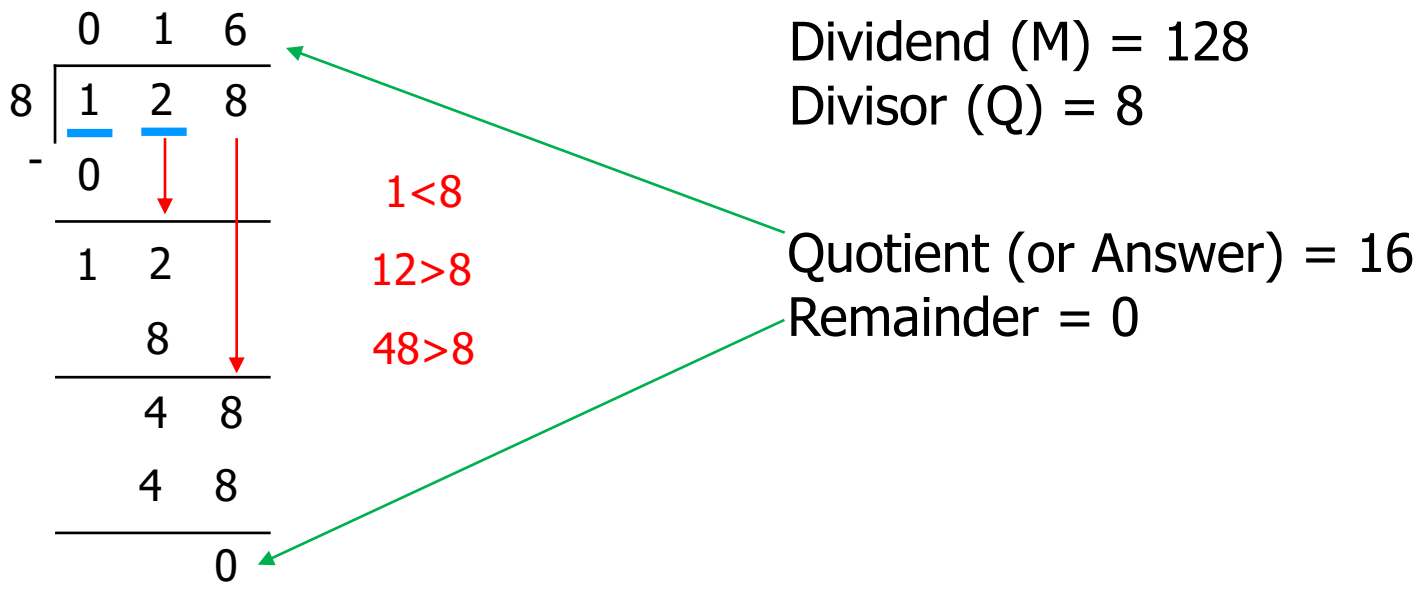
Division

Division

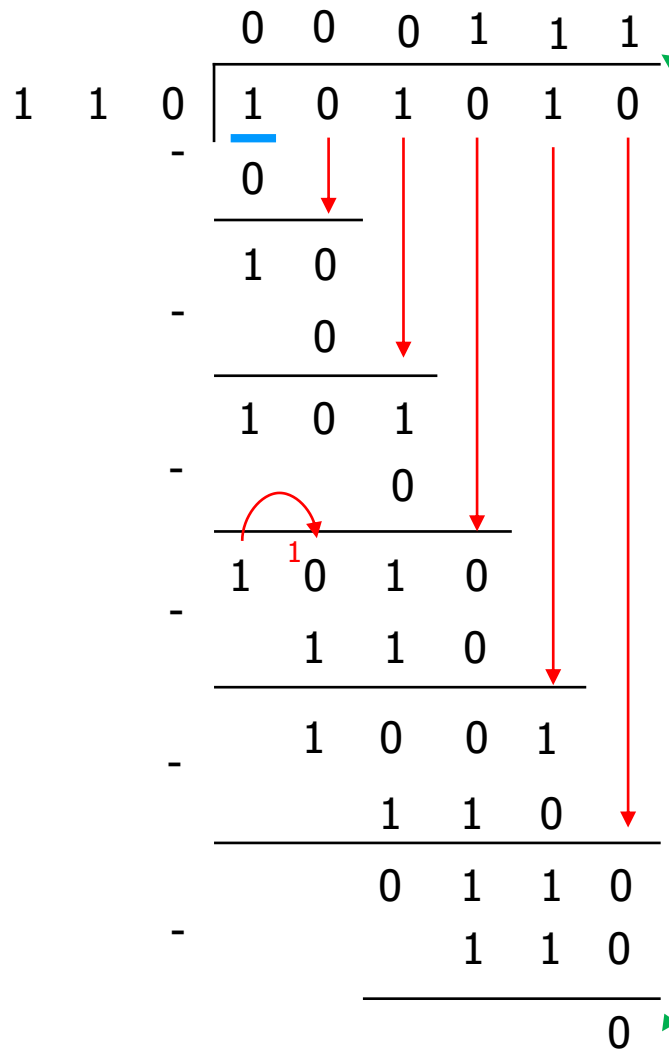
- Division is somewhat more complex than multiplication but is based on the same general principles.
- As before, the basis for the algorithm is the paper-and-pencil approach, and the operation involves repetitive shifting and addition or subtraction.



Long method division



Long method division



Dividend (M) = 101010 $(42)_{10}$

Divisor (Q) = 110 $(6)_{10}$

1<110

10<110

101<110

1010>110

1001>110

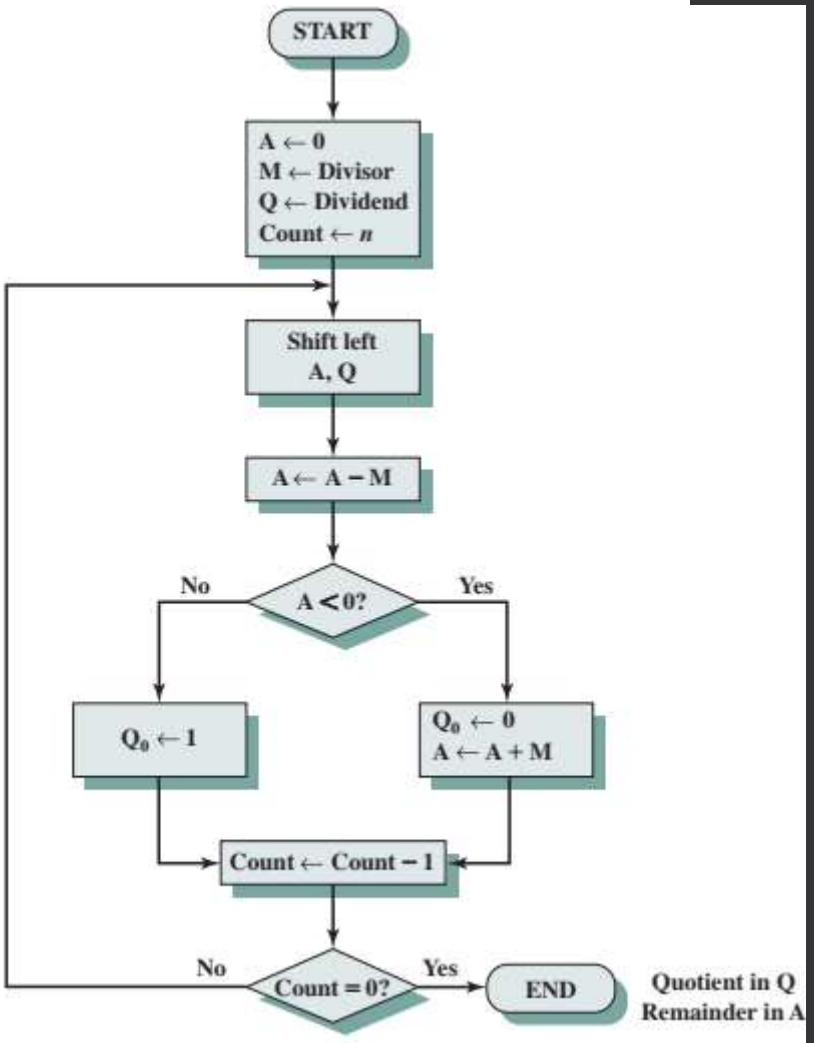
110<110

Quotient (or Answer) = 111

Remainder = 0

Flowchart for Unsigned Binary Division

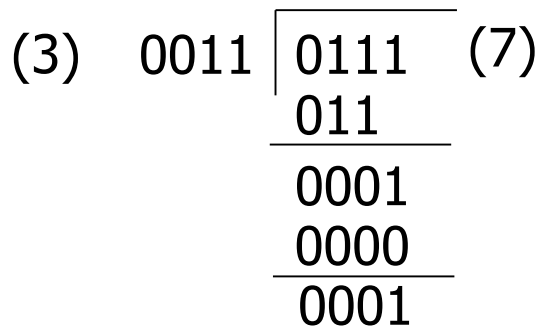
1. Initial A register, Load M&Q register
2. Shift left (A & Q)
3. A – M operation
4. Look at A condition:
   - A<0 = **T** = Q_0 gets a 1 bit
   - A<0 = **F** = Q_0 gets a 0 bit and restores M the previous value



Division in 2s complement

$7 \div 3 = 2$ with 1 remainder (Q ÷ M)

- Q = 0111
- M = 0011



2sM =1101

	A	Q	
	0000	0111	Initial value
	0000	1110	Shift
	1101		Use twos complement of 0011 for subtraction
(-3)	1101		Subtract
	0000	1110	Restore, set $Q_0 = 0$
	0001	1100	Shift
	1101		
(-1)	1110		Subtract
	0001	1100	Restore, set $Q_0 = 0$
	0011	1000	Shift
	1101		
(0)	0000	1001	Subtract, set $Q_0 = 1$
	0001	0010	Shift
	1101		
(-1)	1110		Subtract
	0001	0010	Restore, set $Q_0 = 0$

Q (Dividend) = 7 = 0111
M (Divisor) = 3 = 0011
2sM = 1101

	A	Q	Operation	
	0000	0111	Init.	
	0000	1110	Shift (L)	0000
(-3)	1101		A-M	1101
	0000	1110	Restore A, Q_0 = 0	1101
	0001	1100	Shift (L)	0001
(-1)	1110		A-M	1101
	0001	1100	Restore A, Q_0 = 0	1110
	0011	1000	Shift (L)	0011
(0)	0000		A-M	1101
	0000	1001	Subtract, Q_0 = 1	10000
	0001	0010	Shift (L)	0001
(-1)	1110		A-M	1101
	0001	0010	Restore A, Q_0 = 0	1110

Q = 4 = 0 1 0 0

M = 2 = 0 0 1 0 (2s = 1 1 1 0)

	A	Q	op.
	0 0 0 0	0 1 0 0	init. value
	0 0 0 0	1 0 0 0	shift L
(-2)	1 1 1 0		A - M
	0 0 0 0	1 0 0 0	Restore, $Q_0 = 0$
	0 0 0 1	0 0 0 0	shift L
(-1)	1 1 1 1		A - M
	0 0 0 1	0 0 0 0	Restore, $Q_0 = 0$
	0 0 1 0	0 0 0 0	shift L
(0)	0 0 0 0		A - M
	0 0 0 0	0 0 0 1	subtract, $Q_0 = 1$
	0 0 0 0	0 0 1 0	shift L
(-2)	1 1 1 0		A - M
	0 0 0 0	0 0 1 0	Restore, $Q_0 = 0$

Remainder — Answer

Quick activity #9

Execute the division for the following binary numbers using the long division method and restoring 2s complement division. You must show the full complete process side by side.

1. $4 \div 2$
2. $9 \div 3$
3. $10 \div 4$
4. $11 \div 3$

Outline

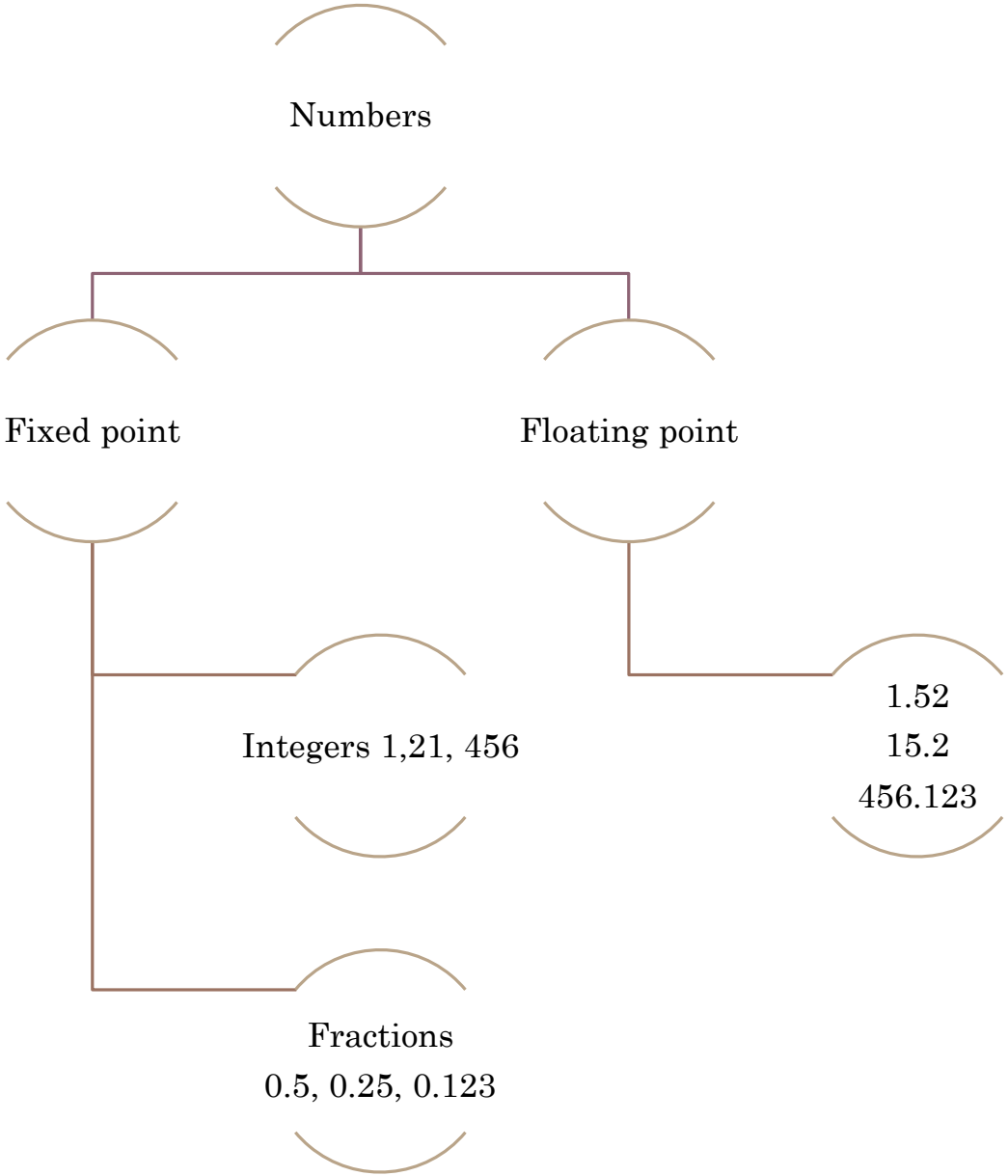
1. Arithmetic & Logic Unit

2. Integer Representation
   - Sign-magnitude
   - Twos complement
   - Conversion between lengths

3. Integer Arithmetic
   - Addition & Subtraction
   - Multiplication
   - Division

4. Floating-point Arithmetic

Introduction



Real numbers

- Numbers with fractions
- Could be done in pure binary
- E.g.:

$$1001.1010 = 2^3 + 2^0 + 2^{-1} + 2^{-3} = 9.625$$

Real numbers

- With a fixed-point notation (e.g., twos complement) it is possible to represent a range of positive and negative integers centered on or near 0.
- **HOWEVER**, this approach has limitations
- Cannot represent a very **LARGE** numbers nor very SMALL fractions.
- For decimals, we overcome this limitation by using scientific notation

976,000,000,000,000	9.76×10^{14}
0.000000000000976	9.76×10^{-14}

Real numbers

- Where is the binary or radix point?
- Fixed?
  - Very limited
- Moving?
  - How do we show where it is?

Floating-point representation

- What about binary numbers?

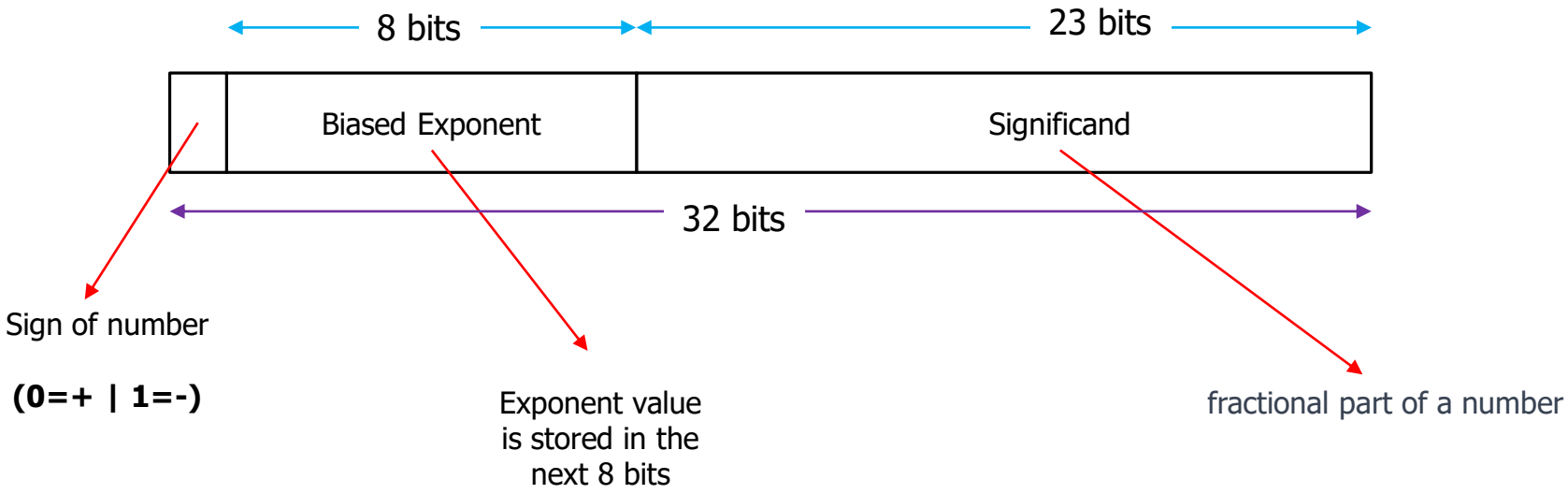
$$\pm S \times B^{\pm E}$$

- Sign: plus, or minus
- Significand S
- Exponent E

Signs for Floating Point

- Significand is stored in **2s complement**
- Exponent is in **biased** notation
  - e.g. Bias of 127 means $(2^{k-1} - 1)$, k is number of bits
    1. **8-bit** exponent field
       1. Pure value range 0-255
       2. Range -127 to +128
    2. Subtract 127 to get the correct value

Floating-point representation



$1.6328125 \times 2^{20} = 1.101001 \times 2^{10100}$

Floating-point representation

$1.6328125 \times 2^{20} = 1.101001 \times 2^{10100}$

1. The original value of exponent = 20

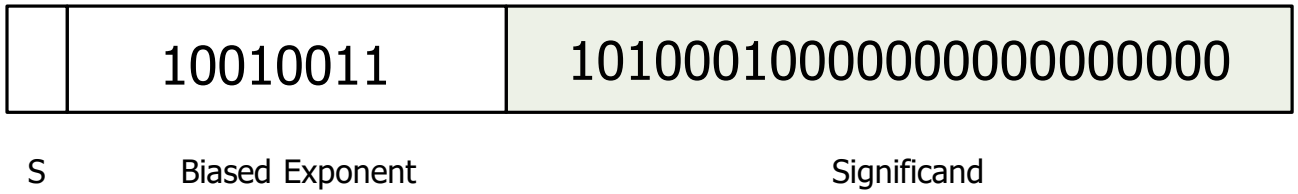
2. To get the exponent value in binary = original exponent + bias exponent

20 + 127 = 147 → 10010011

3. The biased exponent value is = 10010011

Floating-point representation

$1.6328125 \times 2^{20} = 1.1010001 \times 2^{10100}$



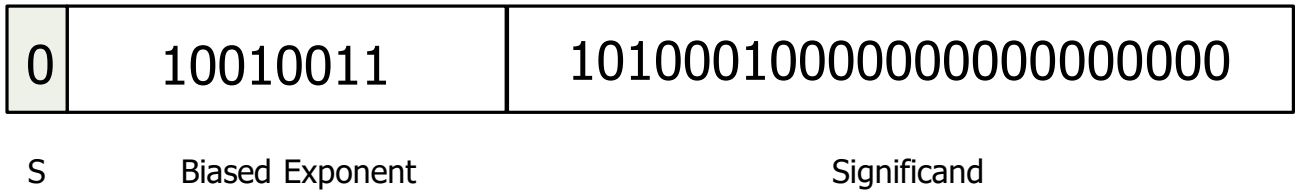
How do we get the significand value?

2. Convert the fractional portion to binary
   0.6328125 to binary = 101001

3. Therefore, the significand is 101001 + remaining 0 to fulfil 23 bits

Floating-point representation

$1.6328125 \times 2^{20} = 1.1010001 \times 2^{10100}$



How do we identify the sign value?

0 = (+) and 1 (-)

$1.1010001 \times 2^{10100} = 0\ 10010011\ 10100010000000000000000 = 1.6328125 \times 2^{20}$

Floating-point representation

Tip: 20 + 127 = 147

$$1.1010001 \times 2^{10100} = 0\ 10010011\ 10100010000000000000000 = 1.6328125 \times 2^{20}$$
$$-1.1010001 \times 2^{10100} = 1\ 10010011\ 10100010000000000000000 = -1.6328125 \times 2^{20}$$
$$1.1010001 \times 2^{-10100} = 0\ 01101011\ 10100010000000000000000 = 1.6328125 \times 2^{-20}$$
$$-1.1010001 \times 2^{-10100} = 1\ 01101011\ 10100010000000000000000 = -1.6328125 \times 2^{-20}$$

Tip: -20 + 127 = 107

2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
128	64	32	16	8	4	2	1
0	1	1	0	1	0	1	1

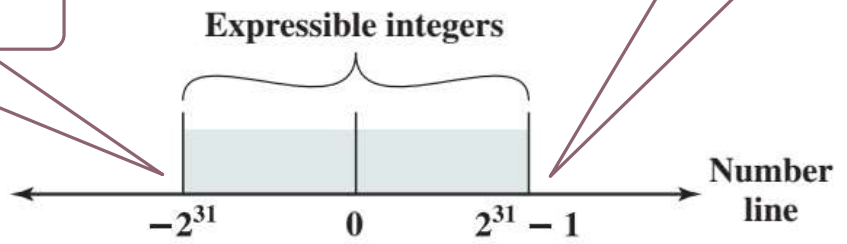
Normalization

- Floating point numbers are usually normalized
  - i.e. exponent is adjusted so that **leading bit (MSB) of significand is 1**
- Since it is always 1 there is **no need to store it**
  - 23 bits can store 24-bit number
- c.f. Scientific notation where numbers are normalized to give a single digit before the decimal point
  - e.g. 3.123×10^3

Expressible Numbers

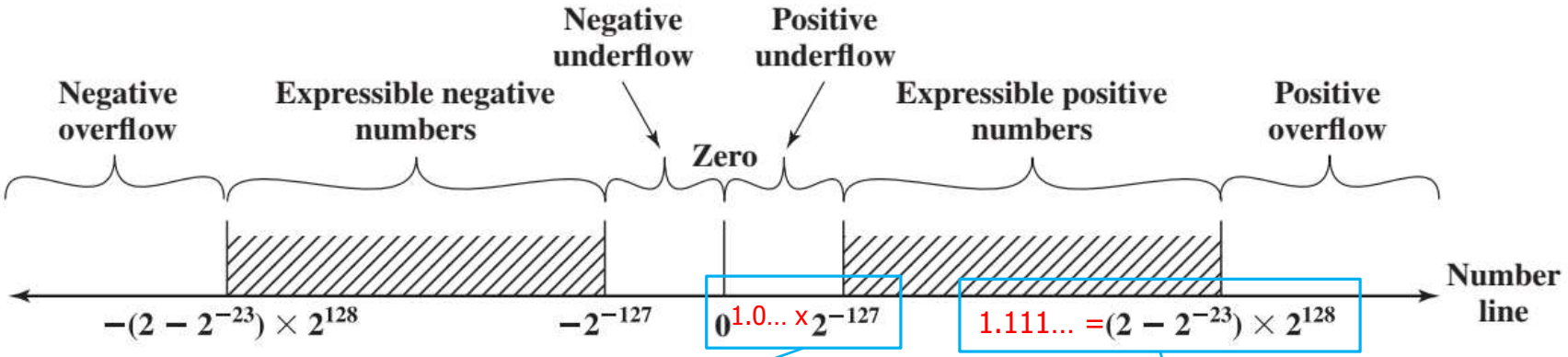
2,147,483,647

-2,147,483,647



(a) Twos complement integers

(32 bits)



(b) Floating-point numbers

The biggest (+) number can be represented

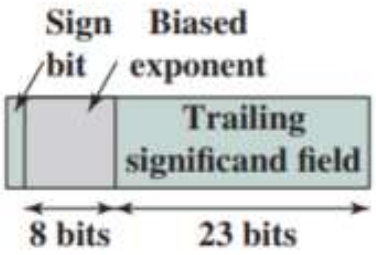
The smallest (+) number can be represented

(1 sign bit, 8 bit exponent, 23-bit significand)

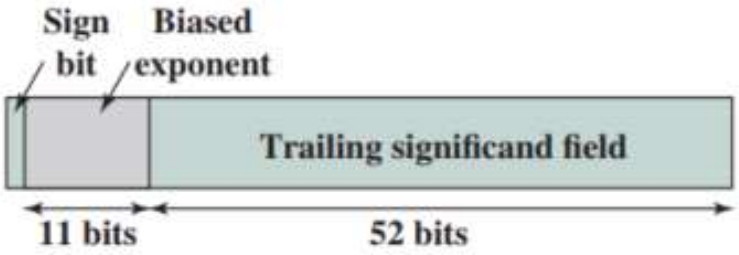
IEEE 754

- The most important floating-point representation is defined in IEEE Standard 754, adopted in 1985 and revised in 2008.
- This standard was developed to facilitate the portability of programs from one processor to another and to encourage the development of sophisticated, numerically oriented programs.
- The standard has been widely adopted and is used on virtually all contemporary processors and arithmetic coprocessors. IEEE 754-2008 covers both binary and decimal floating-point representations.
- Standard for floating point storage
- **32- 64- and 128-bit standards**
- **8, 11 and 15-bit exponent respectively**
- Extended formats (both significand and exponent) for intermediate results

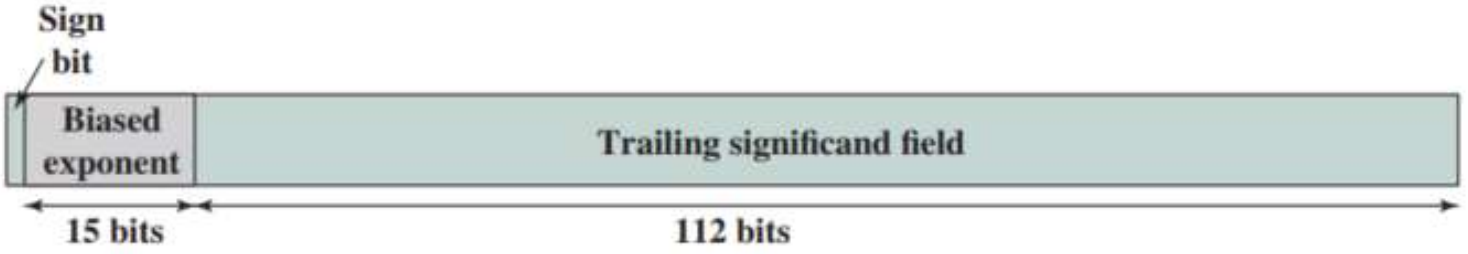
IEEE 754



(a) Binary32 format



(b) Binary64 format



(c) Binary128 format

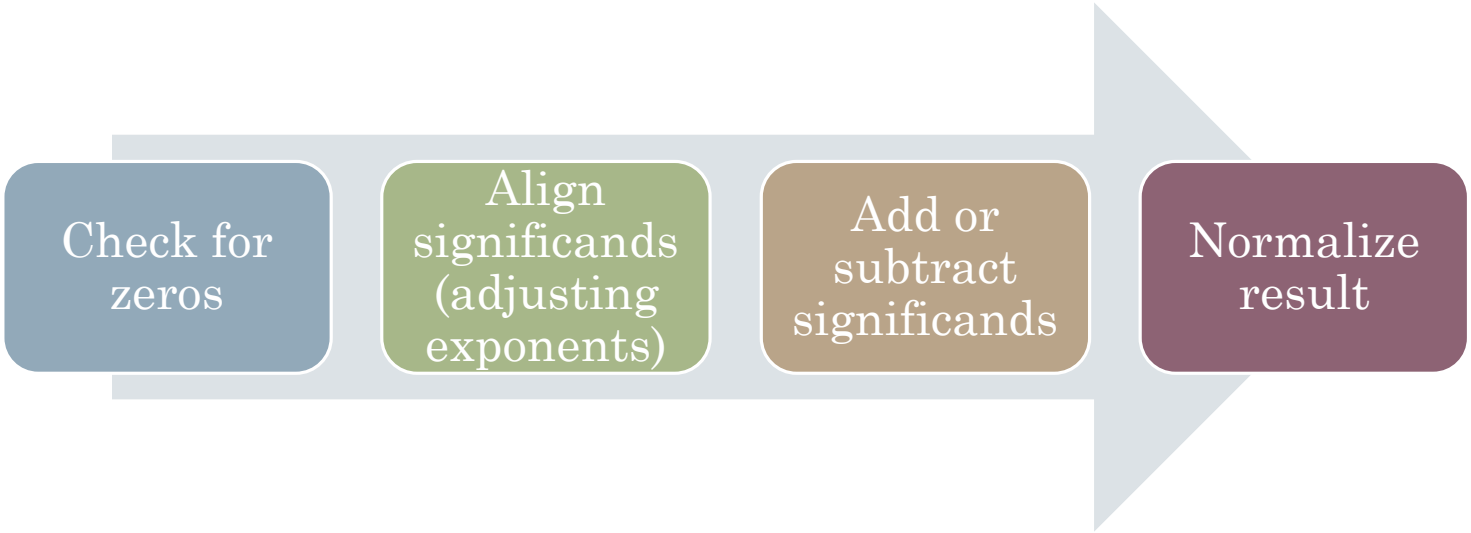
Parameter	Format		
	Binary32	**Binary64**	**Binary128**
Storage width (bits)	32	64	128
Exponent width (bits)	8	11	15
Exponent bias	127	1023	16383
Maximum exponent	127	1023	16383
Minimum exponent	−126	−1022	−16382
Approx normal number range (base 10)	$10^{-38}, 10^{+38}$	$10^{-308}, 10^{+308}$	$10^{-4932}, 10^{+4932}$
Trailing significand width (bits)*	23	52	112
Number of exponents	254	2046	32766
Number of fractions	2^{23}	2^{52}	2^{112}
Number of values	1.98×2^{31}	1.99×2^{63}	1.99×2^{128}
Smallest positive normal number	2^{-126}	2^{-1022}	2^{-16362}
Largest positive normal number	$2^{128} - 2^{104}$	$2^{1024} - 2^{971}$	$2^{16384} - 2^{16271}$
Smallest subnormal magnitude	2^{-149}	2^{-1074}	2^{-16494}

Floating Point Arithmetic

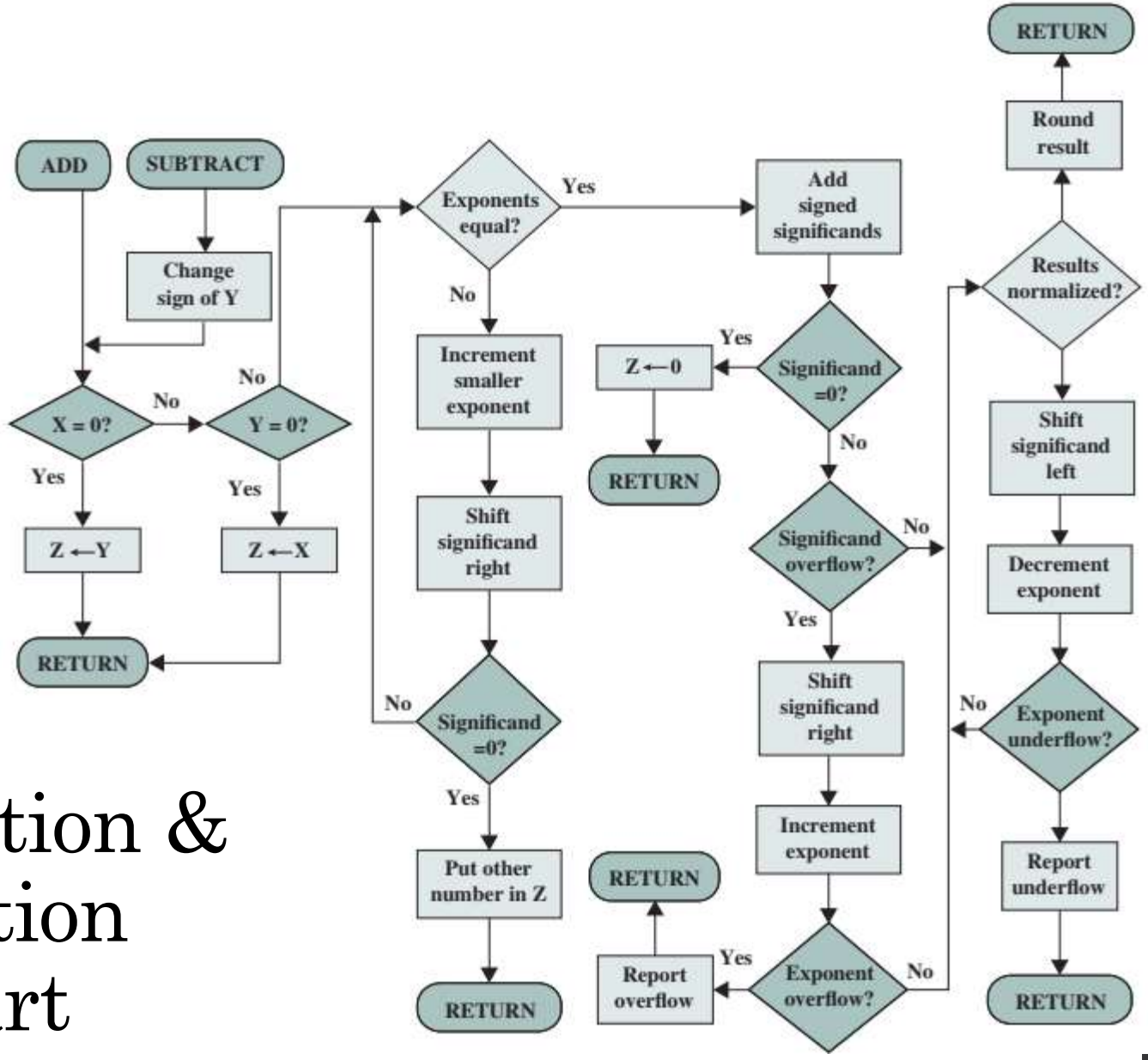
- FP operation may cause
  - Exponent overflow
    - Positive exponent exceed maximum possible value
    - May be + ∞ or - ∞
  - Exponent underflow
    - Negative exponent less than the minimum possible
    - Number too small, may be reported as 0
  - Significand underflow
    - When aligning, digits flow off the right end of significand
  - Significand overflow
    - Carry out of most significant bit
    - E.g. when adding 2 significand of same sign

FP Arithmetic Addition/Subtraction

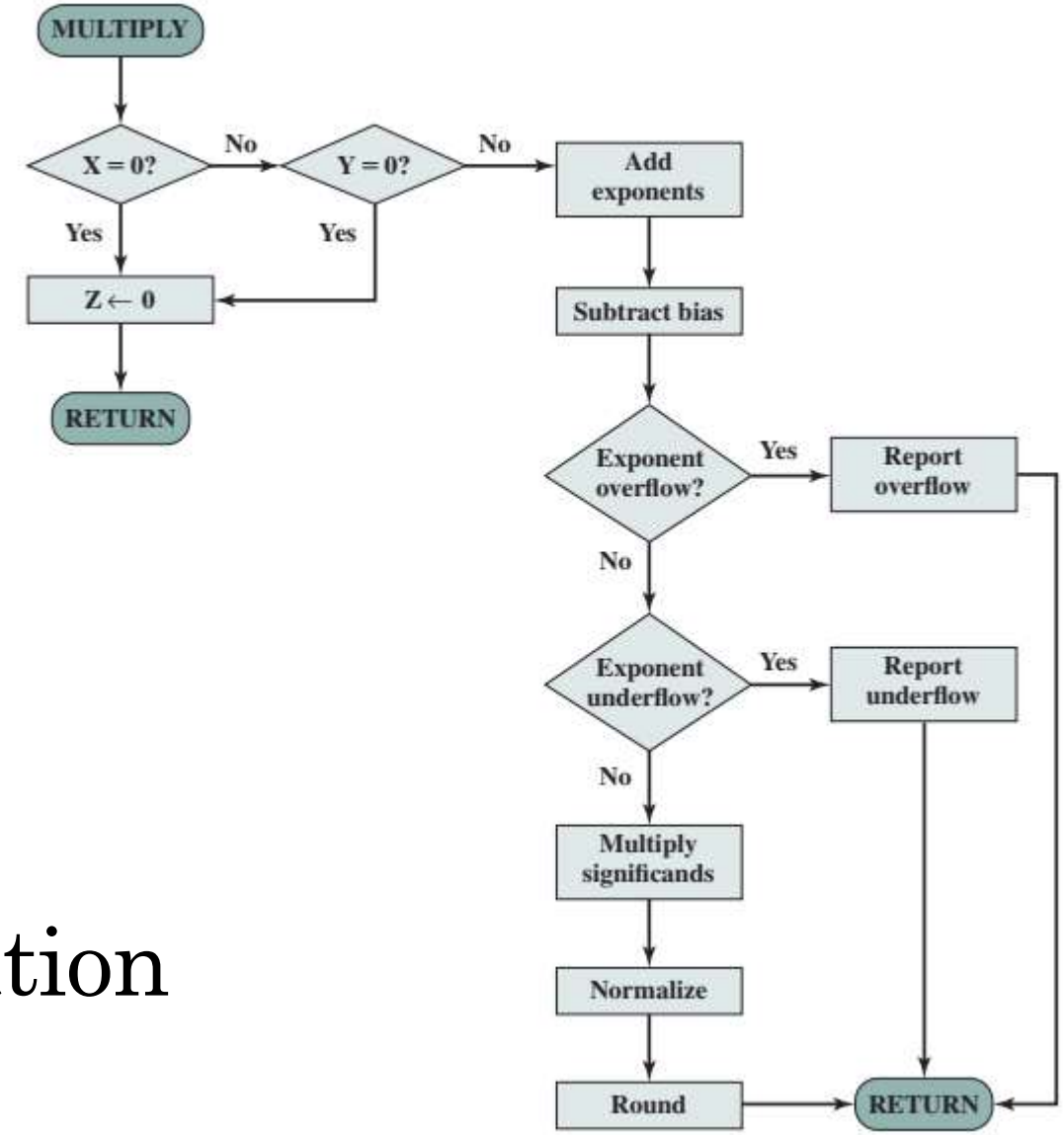
- More complicated than multiplication/division
  - Need for alignment
- 4 phases



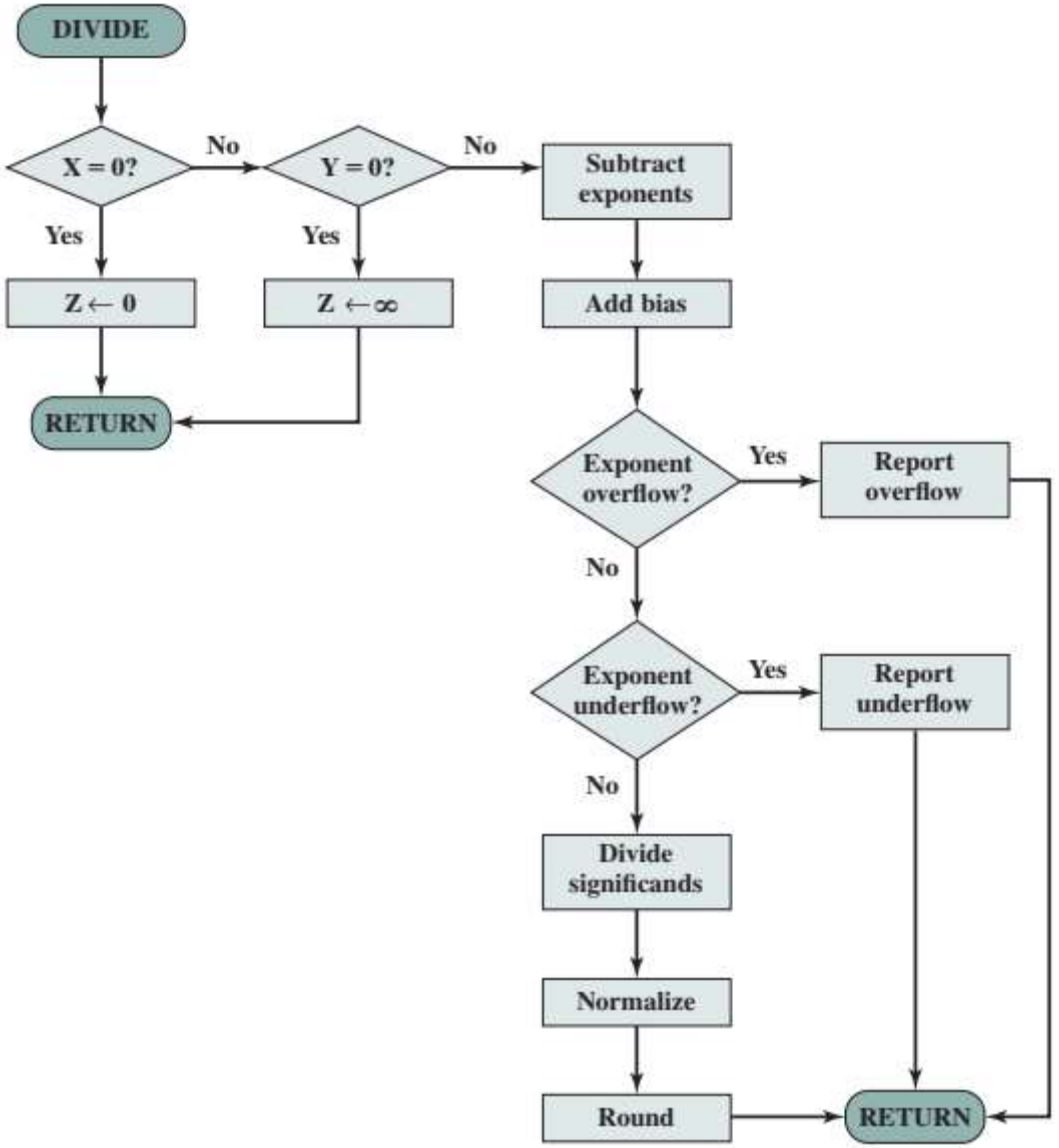
FP Addition & Subtraction Flowchart



FP Multiplication Flowchart



FP Division Flowchart



Quick Activity #10

The following numbers use the IEEE 32-bit floating-point format. What is the equivalent decimal value?

- 0 10000000 10100000000000000000000
- 1 10000010 00100000000000000000000

Quick Activity #11

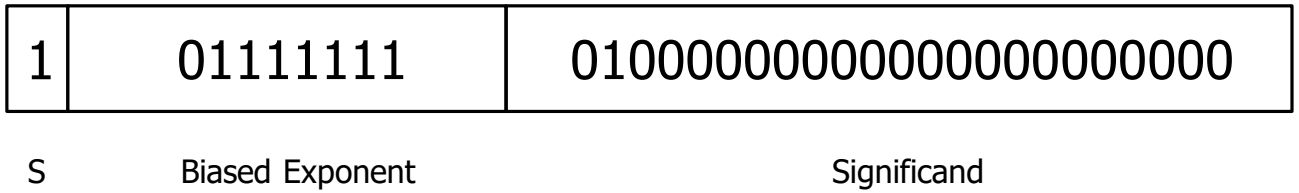
Express the following numbers in IEEE 32-bit floating-point format

- -5
- -6
- 384
- 1/16

Quick Activity #11

Express the following numbers in IEEE 32-bit floating-point format

- -5



1. Determine the sign bit
2. Convert the integer to binary value
3. Determine the exponent
    1. For 32-bit floating point, we will be using 8-bit biased exponent
4. Determine the significand

Reflection…

From the lecture today, I learnt……

Link on eLearn / Chat

Image source: https://www.flickr.com/photos/29143375@N05/4582236533